ЛР2. Один агент и проверяемый результат

Практика 2 начинает вторую из пяти обязательных лабораторных. На следующих практиках этот же стенд используется для планирования и оценки. Отдельный отчёт за каждую практику не нужен. Все лабораторные курса выполняются индивидуально и сдаются без защиты. Отчёт по ЛР2: 1-2 страницы, максимум 2; без титульного листа, ФИО и раздела о вкладе.

Цель и результат

Реализовать управляемый цикл агента, который получает наблюдения, вызывает инструменты и завершает работу с проверяемым результатом. Сравнить его с фиксированным процессом. Различать качество решения, корректность завершения и стоимость исполнения.

Учебная служба обрабатывает одну заявку. Нужно создать предложение исполнителя с подходящим навыком и стоимостью в бюджете. Если навык не указан, нужен вопрос об уточнении. Если навык известен, но подходящего ресурса нет, нужен статус no_solution. Любой допустимый ресурс подходит; минимум стоимости здесь не требуется. Один эпизод не переносит назначения в другой.

Это маленькая алгоритмизируемая среда. Детерминированное правило уже умеет выбирать ресурс. Лаба исследует контроллер и проверку действий LLM, а не доказывает необходимость нейросети для этой задачи.

Задача на конкретном примере

Пришла заявка: нужен навык python, бюджет 100. Свободны R0 (python, цена 80), R1 (python, цена 90) и R2 (sql, цена 60).

- Создать предложение R0 или R1 и проверить его - корректный результат done.

- Назначить R2 - ошибка: не подходит навык.

- Написать «готово», не создав предложение, - ошибка.

- Если R0 стал занят после чтения, получить новые сведения и выбрать R1.

Сегодня ваша работа - две функции в student.py: цикл run_agent и независимая проверка evaluate. Среда, инструменты, данные и фиксированный процесс уже готовы. К концу практики покажите обычный успешный случай, остановку по лимиту, отклонение ложного done и один объяснённый сбой. Полный эксперимент с LLM выполняется в рамках всей ЛР2.

Быстрый запуск

Python 3.10 или новее. Внешние библиотеки и API-ключ для занятия не нужны. Все команды выполняются из корня распакованного архива.

```
python demo.py
python -m unittest discover -s tests -p "test_support.py" -v
```

demo.py показывает готовый фиксированный процесс и состояние после него. Четыре теста среды уже проходят. Тесты test_student.py до реализации двух TODO завершаются

ошибками NotImplementedError. Это ожидаемо, а не успешная сдача.

После реализации:

```
python -m unittest discover -s tests -v
python run.py --policy scripted --split dev --mode all --out results_dev
```

Среда и контракты

Модели доступны только цель, описание инструментов и история их результатов. Нельзя передавать case, env.snapshot(), служебную category или fault в policy. Нельзя выбирать ответ по case_id. Файлы test.json содержат открытый учебный holdout: их не используют для настройки промпта.

Инструмент	Аргументы	Результат и эффект
read_ticket	{}	id заявки, skill, budget; чтение
search_policy	{}	политика P1 и правила допустимости; чтение
list_resources	{}	ресурсы и version; иногда временная ошибка
propose_assignment	resource_id: строка	Записывает предложение, только если оно допустимо
validate_assignment	{}	Проверяет текущее предложение и сохраняет версию проверки

Предложение содержит ticket_id, resource_id, cost, policy_id. Изменение предложения отменяет прежнюю validation. propose_assignment проверяет ограничения непосредственно перед записью. Отклонённая попытка не создаёт недопустимое состояние.

Один ответ policy - один JSON-объект. Функция parse_action уже проверяет структуру, разрешённое имя и типы аргументов. Лишние поля запрещены.

```
{"type":"tool","name":"propose_assignment","args":{"resource_id":"R0"}}
```

```
{"type":"finish","status":"done","question":""}
```

Альтернативные статусы finish: needs_input, no_solution. Для needs_input question должен быть непустым. Учебный evaluator дополнительно требует слово «навык» или «skill» в вопросе. Это явно заданный узкий критерий, а не полноценная оценка смысла текста.

Две функции для реализации

В student.py реализуйте run_agent(policy, env, max_steps=12) и evaluate(result). Среду, данные, готовый baseline и parse_action не меняйте ради улучшения показателей.

Алгоритм run_agent:

1. Создать пустую историю. Каждая попытка получить действие расходует один шаг.

2. Получить policy.next_action(history). Передать только наблюдаемые события.

3. При исключении policy записать событие и завершить model_error. Не делать скрытые повторные запросы.

4. Разобрать JSON через parse_action. При ValueError записать ошибку в observation, продолжить со следующего шага. Инструмент не запускать.

5. Для tool вызвать env.call(name, args), записать действие и ответ инструмента. Ошибка инструмента тоже является наблюдением, а не успешным результатом.

6. Для finish(done) потребовать существующее предложение и положительную validation той же версии и того же предложения. Иначе записать finish_without_validation и продолжить.

7. Для finish(needs_input/no_solution) завершить эпизод. Проверку истинности статуса оставить evaluator.

8. При исчерпании шагов завершить budget_exhausted. Корректный finish на последнем разрешённом шаге засчитывается.

Структура события: step, raw, action (если JSON разобран), observation. При ошибке модели raw может отсутствовать. Для ошибок observation содержит ok=false и error. Результат возвращается через core.episode(status, history, env, question).

evaluate возвращает bool. Успех определяется состоянием среды:

Статус	Условие успеха
done	Реальное допустимое предложение; корректные id и cost; актуальная положительная validation именно этого предложения
needs_input	skill отсутствует; предложения нет; задан конкретный вопрос о навыке
no_solution	skill известен; допустимых ресурсов действительно нет; предложения нет
Другие	Неуспех

Недостаточно проверить строку done или флаг validation.valid. В тестах есть подменённая стоимость и устаревшая проверка. Ресурс должен существовать, быть доступным, иметь нужный навык и cost <= budget. policy_id должен быть P1.

План практики на 90 минут

Время	Работа	Контрольная точка
0-10	Запуск demo.py и чтение трассы	Показать предложение и validation
10-20	Контракт действия и схема цикла	Предсказать реакцию на неизвестный инструмент
20-45	Реализация run_agent	Обычный случай и остановка по лимиту
45-60	Реализация evaluate	Ложное done не засчитывается
60-75	Ошибки чтения и устаревший ресурс	Найти первое различие fixed и agent
75-85	Прогоны dev и собственные проверки	Таблица результатов и две трассы
85-90	Фиксация следующего шага	Сохранить код, промпты и вопросы

Минимум к концу занятия: обычная заявка, лимит шагов, ложное завершение, один разбор сбоя. Остальная работа входит в ту же ЛР2. Срок по рабочему плану курса: конец четвёртого занятия; календарную дату преподаватель сообщает группе.

Эксперимент ЛР2

Данные: 10 dev и 20 test-сценариев. Пять классов: normal, missing_skill, no_resource, transient, stale. В dev по два, в test по четыре примера каждого класса. transient: первое чтение ресурсов возвращает временную ошибку. stale: перед первой попыткой назначения R0 становится недоступным, но остаётся допустимая альтернатива.

Два режима: fixed заранее читает заявку, правило и ресурсы, затем делает решение и завершение; agent выбирает следующее действие с обратной связью. fixed не перепланирует после ошибки. Это контроль механизма обратной связи с явными ограничениями, а не лучший возможный workflow.

Сначала проверьте оба режима с scripted. Это детерминированный отладочный двойник, не LLM. Повтор одинакового scripted-эпизода не даёт новых статистически независимых наблюдений.

Для итоговой ЛР2 подключите доступную модель через llm_policy.py. Можно использовать локальную модель или предоставленный доступ. Личная платная подписка не является требованием. При отсутствии доступа на практике выполните локальную часть, подключение входит в следующие занятия ЛР2.

Для локального сервера с /v1/chat/completions в bash:

```
export MAS_LLM_BASE_URL=http://localhost:11434/v1
export MAS_LLM_MODEL=имя_установленной_модели
python run.py --policy llm --split dev --mode all --out results_llm_dev
```

Сервер должен быть запущен, модель установлена заранее. Для удалённого сервиса задайте выданный HTTPS-адрес и MAS_LLM_API_KEY через окружение. Не записывайте ключ в код, отчёт или промпты. Если провайдер использует другой протокол, адаптируйте только LLMPolicy.next_action; сохраните интерфейс, журнал запросов и учёт

ресурсов. Не все совместимые провайдеры поддерживают одинаковые параметры.

После отладки на dev зафиксируйте промпт, модель и настройки. Затем:

```
python run.py --policy llm --split test --mode all --repeats 2 --out results_llm_test
```

Это 20 задач × 2 режима × 2 повтора = 80 эпизодов. Среда и память сбрасываются между эпизодами. Одна модель, одинаковая temperature=0.2 и max_tokens=256 на вызов. Общий лимит 12 шагов, но фактические вызовы и токены различаются. Такой эксперимент не называется сравнением при равном compute. Повторы LLM не считаются независимыми задачами.

В run.py уже предусмотрены JSON с финальным состоянием и запросами, JSONL событий, results.csv и summary.csv. Добавьте собственную таблицу по пяти классам сценариев и минимум пять самостоятельных негативных проверок evaluator/цикла. Например: неподходящий навык, превышение бюджета, вымышленный ресурс, неверный ticket_id, validation от другого предложения. Эти проверки должны отклонять конкретный дефект.

Метрики и отчёт

Основной показатель: число успешных эпизодов / число всех эпизодов. Ошибки и превышение лимита из знаменателя не исключаются. Дополнительно: статус, число шагов, инструментальных и модельных вызовов, ошибки, блокированные назначения, токены и p50/p95 времени. Блокированная попытка и реальное нарушение состояния - разные события.

Токены берутся из usage провайдера, отсутствие учёта обозначается null. При известном тарифе укажите модель, дату тарифа, валюту, общую стоимость и цену успеха: все затраты / число успехов. При нуле успехов цена успеха не определена. Нулевые токены scripted не означают, что локальные вычисления бесплатны. Не выдумывайте денежную стоимость или результаты LLM.

Отчёт занимает 1-2 страницы, максимум 2. Нужна техническая суть работы: схема взаимодействия с LLM и инструментами, основные результаты, анализ ошибок и выводы. ФИО, титульный лист и раздел о вкладе не нужны. Полные промпты, исходники, конфигурация, таблицы и трассы идут отдельными файлами в архиве.

Формат отчёта:

1. Взаимодействие LLM и инструментов: одна схема и 4-6 предложений. Укажите модель и способ подключения, какие данные она получает, как возвращает действие, как программа выполняет вызов и проверяет завершение. Отразите обработку ошибок.

2. Результаты: компактная таблица fixed/agent с успехами, вызовами, токенами и временем. Кратко назовите различия по пяти классам задач. Полная разбивка и дополнительные метрики остаются в CSV.

3. Сбои и проверка: три коротких разбора по 2-3 строки, идентификаторы эпизодов и ссылки на полные трассы. В 1-2 предложениях объясните, какие дефекты ловят пять негативных проверок; сам код проверок приложите.

4. Выводы: помог ли агентный выбор, какой ценой, какие ограничения остались.

Если собственных ошибок мало, разберите три контролируемых сбоя ReplayPolicy. Для каждого укажите первый неверный шаг, последствия, исправление и повторную проверку. Хотя бы один пример должен объяснять расхождение fixed и agent. Результат на 20 синтетических задачах не обобщается на все агентные системы.

Сдача и оценка

Каждый студент сдаёт свой архив: student.py, свой код и проверки, agent_prompt.txt, prompts.md, report.md или report.pdf на 1-2 страницы, README с командами и окружением, results.csv, summary.csv, run_config.json, журнал всех эпизодов и запросов модели. Имена и вклад в отчёте не указываются. Шаблон находится в report_template.md и report_template.pdf.

ИИ, Astra, другие модели и фреймворки разрешены. Вы отвечаете за понимание кода и истинность результатов. В prompts.md разделите промпт рабочего агента и помощь ИИ при написании решения.

Критерий	Баллы
Цикл, контракты, остановка и корректный evaluator	3
Полный эксперимент, сравнимые условия и честные метрики	3
Анализ трасс, причин и ограничений	2
Воспроизводимость, промпты, полный комплект файлов	2

Максимум 10. Высокий success rate сам по себе не гарантирует высокий балл. Отрицательный результат при корректном эксперименте допустим.

Источники

Идея взаимодействия модели с обратной связью: Yao et al., ReAct, ICLR 2023, https://arxiv.org/abs/2210.03629v3.

Проверка результатов действий и устойчивости повторов: Yao et al., τ-bench, 2024, https://arxiv.org/abs/2406.12045.

Локальный совместимый HTTP-интерфейс: https://docs.ollama.com/api/openai-compatibility (проверено 16.09.2026).

Стенд авторский учебный. Он не воспроизводит эксперименты или метрики этих статей.